

```

1  import flet as ft
2  import paho.mqtt.client as mqtt
3  import random
4  import datetime
5  import threading
6  import time
7  import json
8  import os
9  import tempfile # Garante um diretório com permissão de escrita em qualquer SO
10
11  # --- CONFIGURAÇÕES MQTT ---
12  MQTT_SERVER = "emqx.ifspb-czemnumeros.com.br"
13  MQTT_PORT = 1883
14  MQTT_USER = "esp12_01"
15  MQTT_PASS = "esp12_01"
16
17  # Dicionário global para controlar o estado das automações
18  automacoes = {
19      "casa/rele1": {"timer_minutos": 0, "hora_agenda": "18:00", "dias_agenda": set(),
20      "timer_objeto": None},
21      "casa/rele2": {"timer_minutos": 0, "hora_agenda": "18:00", "dias_agenda": set(),
22      "timer_objeto": None}
23  }
24
25  # --- DEFINIÇÃO DO CAMINHO COMPATÍVEL MULTIPLATAFORMA ---
26  # tempfile.gettempdir() aponta para uma pasta com permissões totais tanto no PC
27  # quanto no Android/iOS
28  CONFIG_FILE = os.path.join(tempfile.gettempdir(), "config_automacoes_jardim.json")
29  print(f"DEBUG: Caminho seguro utilizado = {CONFIG_FILE}")
30
31  def main(page: ft.Page):
32      page.title = "Meu Jardim - Automação"
33      page.theme_mode = ft.ThemeMode.DARK
34      page.window_width = 400
35      page.window_height = 700
36      page.horizontal_alignment = "center"
37
38      # --- BARRA SUPERIOR ---
39      page.appbar = ft.AppBar(
40          title=ft.Text("🏠 AUTOMAÇÃO", size=22, weight="bold"),
41          center_title=True,
42          bgcolor="surface",
43      )
44
45      # --- STATUS DO BROKER ---
46      status_led = ft.Container(width=12, height=12, bgcolor="red", border_radius=6)
47      status_text = ft.Text("Desconectado", color="red", weight="bold")
48
49      # --- CLIENTE MQTT ---
50      client_id = f'flet-mqtt-{random.randint(0, 1000)}'
51      mqttc = mqtt.Client(client_id=client_id)
52      mqttc.username_pw_set(MQTT_USER, MQTT_PASS)
53
54      def on_connect(client, userdata, flags, rc):
55          if rc == 0:
56              status_led.bgcolor = "green"
57              status_text.value = "Online"
58              status_text.color = "green"
59          else:
60              status_led.bgcolor = "red"
61              status_text.value = f"Erro {rc}"
62              status_text.color = "red"
63          page.update()
64
65      mqttc.on_connect = on_connect
66
67      # --- FUNÇÃO ATUADORA (MUDAR RELE) ---
68      def mudar_rele(e):
69          topic = e.control.data
70          msg = "ON" if e.control.value else "OFF"
71
72          if msg == "OFF" and automacoes[topic]["timer_objeto"] is not None:
73              try:

```

```

71         automacoes[topic]["timer_objeto"].cancel()
72         automacoes[topic]["timer_objeto"] = None
73     except Exception:
74         pass
75
76     try:
77         mqttc.publish(topic, msg)
78     except Exception as err:
79         print(f"Erro MQTT: {err}")
80
81 # --- PERSISTÊNCIA LOCAL ---
82 def salvar_dados_no_disco():
83     try:
84         dados_para_salvar = {
85             "casa/rele1": {
86                 "timer_minutos": automacoes["casa/rele1"]["timer_minutos"],
87                 "hora_agenda": automacoes["casa/rele1"]["hora_agenda"],
88                 "dias_agenda": list(automacoes["casa/rele1"]["dias_agenda"])
89             },
90             "casa/rele2": {
91                 "timer_minutos": automacoes["casa/rele2"]["timer_minutos"],
92                 "hora_agenda": automacoes["casa/rele2"]["hora_agenda"],
93                 "dias_agenda": list(automacoes["casa/rele2"]["dias_agenda"])
94             }
95         }
96         with open(CONFIG_FILE, "w") as f:
97             json.dump(dados_para_salvar, f)
98             print("DEBUG: Dados guardados em segurança!")
99     except Exception as e:
100         print(f"Erro ao salvar dados: {e}")
101
102 # --- LÓGICA DO TEMPORIZADOR (TIMER) ---
103 def desligar_por_timeout(pino_rele, switch_componente):
104     try:
105         mqttc.publish(pino_rele, "OFF")
106         switch_componente.value = False
107         automacoes[pino_rele]["timer_objeto"] = None
108         automacoes[pino_rele]["timer_minutos"] = 0
109
110         switch_componente.page.snack_bar = ft.SnackBar(ft.Text(f"🔌 {pino_rele}
111         desligado! Tempo terminou."))
112         switch_componente.page.snack_bar.open = True
113         switch_componente.page.update()
114     except Exception as e:
115         print(f"Erro no timeout do timer: {e}")
116
117 def acionar_temporizador(pino_rele, txt_timer, switch_componente):
118     try:
119         minutos = int(txt_timer.value)
120     except ValueError:
121         switch_componente.page.snack_bar = ft.SnackBar(ft.Text("Insira um número
122         válido de minutos!"))
123         switch_componente.page.snack_bar.open = True
124         switch_componente.page.update()
125         return
126
127     if minutos <= 0:
128         switch_componente.page.snack_bar = ft.SnackBar(ft.Text("O tempo deve ser
129         maior que zero!"))
130         switch_componente.page.snack_bar.open = True
131         switch_componente.page.update()
132         return
133
134     if automacoes[pino_rele]["timer_objeto"] is not None:
135         automacoes[pino_rele]["timer_objeto"].cancel()
136
137     mqttc.publish(pino_rele, "ON")
138     switch_componente.value = True
139
140     switch_componente.page.snack_bar = ft.SnackBar(ft.Text(f"🕒 Temporizador
141     iniciado por {minutos} minuto(s)"))
142     switch_componente.page.snack_bar.open = True
143     switch_componente.page.update()

```

```

140
141     segundos = minutos * 60
142     t = threading.Timer(segundos, desligar_por_timeout, args=[pino_rele,
143         switch_componente])
144     automacoes[pino_rele]["timer_objeto"] = t
145     automacoes[pino_rele]["timer_minutos"] = minutos
146     t.start()
147
148     salvar_dados_no_disco()
149
150     # --- INPUTS VISUAIS ---
151     txt_timer_d6 = ft.TextField(label="Minutos", value="30", width=90,
152         text_align="center", dense=True)
153     txt_hora_d6 = ft.TextField(label="HH:MM", value="18:00", width=90,
154         text_align="center", dense=True)
155     buttons_d6 = {}
156
157     txt_timer_d7 = ft.TextField(label="Minutos", value="30", width=90,
158         text_align="center", dense=True)
159     txt_hora_d7 = ft.TextField(label="HH:MM", value="18:00", width=90,
160         text_align="center", dense=True)
161     buttons_d7 = {}
162
163     switch_luz = ft.Switch(value=False, data="casa/rele1", on_change=mudar_rele)
164     switch_tomada = ft.Switch(value=False, data="casa/rele2", on_change=mudar_rele)
165
166     # --- CARREGA DADOS DO ARQUIVO ANTES DE CRIAR INTERFACE ---
167     def carregar_dados_local():
168         try:
169             if os.path.exists(CONFIG_FILE):
170                 with open(CONFIG_FILE, "r") as f:
171                     dados_dict = json.load(f)
172                     print(f"DEBUG: Dados carregados: {dados_dict}")
173
174                     for pino in ["casa/rele1", "casa/rele2"]:
175                         if pino in dados_dict:
176                             automacoes[pino]["hora_agenda"] =
177                                 dados_dict[pino].get("hora_agenda", "18:00")
178                             automacoes[pino]["timer_minutos"] =
179                                 dados_dict[pino].get("timer_minutos", 0)
180                             dias_carregados = dados_dict[pino].get("dias_agenda", [])
181                             automacoes[pino]["dias_agenda"] = set(dias_carregados)
182         except Exception as e:
183             print(f"Erro ao carregar dados: {e}")
184
185     carregar_dados_local()
186
187     # --- CONSTRUTOR DE BLOCOS ---
188     # --- CONSTRUTOR DE BLOCOS ---
189     def criar_bloco_automacao(pino_rele, txt_timer, txt_hora, buttons_dict,
190         switch_componente):
191         def salvar_agenda(e):
192             automacoes[pino_rele]["hora_agenda"] = txt_hora.value
193             salvar_dados_no_disco()
194             switch_componente.page.snack_bar = ft.SnackBar(ft.Text(f"Agenda de
195                 {pino_rele} salva para as {txt_hora.value}!"))
196             switch_componente.page.snack_bar.open = True
197             switch_componente.page.update()
198
199         def alternar_dia_btn(e):
200             container_alvo = e.control.content
201             dia = container_alvo.data
202             texto_alvo = container_alvo.content
203
204             if dia in automacoes[pino_rele]["dias_agenda"]:
205                 automacoes[pino_rele]["dias_agenda"].discard(dia)
206                 container_alvo.bgcolor = "transparent"
207                 container_alvo.border = ft.Border(
208                     ft.BorderSide(1, "white30"), ft.BorderSide(1, "white30"),
209                     ft.BorderSide(1, "white30"), ft.BorderSide(1, "white30")
210                 )
211                 texto_alvo.color = "white"
212             else:

```

[illegible]

```

275         txt_hora,
276         ft.IconButton(icon=ft.Icons.SAVE, icon_color="blue",
                        on_click=salvar_agenda)
277     ], alignment="spaceBetween"),
278
279     ft.Text("Dias da semana:", size=12, color="white60"),
280     ft.Row(lista_botoes, wrap=True, alignment="center", spacing=5) #
    Removido container com bug de tamanho fixo
281 ], spacing=10)
282 )
283 ]
284 )
285
286
287 # --- MONTAGEM DOS CARDS ---
288 card_luz = ft.Card(
289     content=ft.Container(
290         padding=12,
291         content=ft.Column([
292             ft.Row([
293                 ft.Row([ft.Icon(ft.Icons.LIGHTBULB, color="amber", size=28),
294                     ft.Text("Luz (D6)", size=18, weight="w500")]),
295                 switch_luz
296             ], alignment="spaceBetween"),
297             criar_bloco_automacao("casa/rele1", txt_timer_d6, txt_hora_d6,
298                 buttons_d6, switch_luz)
299         ])
300     )
301
302 card_tomada = ft.Card(
303     content=ft.Container(
304         padding=12,
305         content=ft.Column([
306             ft.Row([
307                 ft.Row([ft.Icon(ft.Icons.POWER, color="blueGrey200", size=28),
308                     ft.Text("Tomada (D7)", size=18, weight="w500")]),
309                 switch_tomada
310             ], alignment="spaceBetween"),
311             criar_bloco_automacao("casa/rele2", txt_timer_d7, txt_hora_d7,
312                 buttons_d7, switch_tomada)
313         ])
314     )
315
316 # --- RENDERIZAÇÃO DA INTERFACE ---
317 # Colocamos tudo dentro de uma Column com scroll para nunca quebrar no Android
318 conteudo_principal = ft.Column(
319     controls=[
320         ft.Container(height=15),
321         ft.Row([status_led, status_text], alignment="center"),
322         ft.Divider(),
323         card_luz,
324         card_tomada
325     ],
326     scroll=ft.ScrollMode.AUTO,
327     expand=True
328 )
329
330 page.add(conteudo_principal)
331 page.update()
332
333 # --- PROCESSAMENTO ASSÍNCRONO EM BACKGROUND THREAD ---
334 def conectar_mqtt_e_agendar():
335     try:
336         mqttc.connect(MQTT_SERVER, MQTT_PORT, 60)
337         mqttc.loop_start()
338     except Exception:
339         pass
340
341 def loop_verificacao_horario():
342     while True:

```

```

342     agora = datetime.datetime.now()
343     dias_map = ["Seg", "Ter", "Qua", "Qui", "Sex", "Sáb", "Dom"]
344     dia_atual_str = dias_map[agora.weekday()]
345     hora_atual_str = agora.strftime("%H:%M")
346
347     for pino, dados in automacoes.items():
348         if dados["hora_agenda"] == hora_atual_str and dia_atual_str in
           dados["dias_agenda"]:
349             mqttc.publish(pino, "ON")
350             if pino == "casa/rele1":
351                 switch_luz.value = True
352             if pino == "casa/rele2":
353                 switch_tomada.value = True
354             page.update()
355
356 import flet as ft
357 import paho.mqtt.client as mqtt
358 import random
359 import datetime
360 import threading
361 import time
362 import json
363 import os
364 import tempfile # Garante um diretório com permissão de escrita em qualquer SO
365
366 # --- CONFIGURAÇÕES MQTT ---
367 MQTT_SERVER = "emqx.ifspb-czemnumeros.com.br"
368 MQTT_PORT = 1883
369 MQTT_USER = "esp12_01"
370 MQTT_PASS = "esp12_01"
371
372 # Dicionário global para controlar o estado das automações
373 automacoes = {
374     "casa/rele1": {"timer_minutos": 0, "hora_agenda": "18:00", "dias_agenda": set(),
375                   "timer_objeto": None},
376     "casa/rele2": {"timer_minutos": 0, "hora_agenda": "18:00", "dias_agenda": set(),
377                   "timer_objeto": None}
378 }
379
380 # --- DEFINIÇÃO DO CAMINHO COMPATÍVEL MULTIPLATAFORMA ---
381 # tempfile.gettempdir() aponta para uma pasta com permissões totais tanto no PC
382 # quanto no Android/iOS
383 CONFIG_FILE = os.path.join(tempfile.gettempdir(), "config_automacoes_jardim.json")
384 print(f"DEBUG: Caminho seguro utilizado = {CONFIG_FILE}")
385
386 def main(page: ft.Page):
387     page.title = "Meu Jardim - Automação"
388     page.theme_mode = ft.ThemeMode.DARK
389     page.window_width = 400
390     page.window_height = 700
391     page.horizontal_alignment = "center"
392
393     # --- BARRA SUPERIOR ---
394     page.appbar = ft.AppBar(
395         title=ft.Text("🏠 AUTOMAÇÃO", size=22, weight="bold"),
396         center_title=True,
397         bgcolor="surface",
398     )
399
400     # --- STATUS DO BROKER ---
401     status_led = ft.Container(width=12, height=12, bgcolor="red", border_radius=6)
402     status_text = ft.Text("Desconectado", color="red", weight="bold")
403
404     # --- CLIENTE MQTT ---
405     client_id = f'flet-mqtt-{random.randint(0, 1000)}'
406     mqttc = mqtt.Client(client_id=client_id)
407     mqttc.username_pw_set(MQTT_USER, MQTT_PASS)
408
409     def on_connect(client, userdata, flags, rc):
410         if rc == 0:
411             status_led.bgcolor = "green"
412             status_text.value = "Online"
413             status_text.color = "green"
414         else:

```

```

411         status_led.bgcolor = "red"
412         status_text.value = f"Erro {rc}"
413         status_text.color = "red"
414     page.update()
415
416     mqttc.on_connect = on_connect
417
418     # --- FUNÇÃO ATUADORA (MUDAR RELE) ---
419     def mudar_rele(e):
420         topic = e.control.data
421         msg = "ON" if e.control.value else "OFF"
422
423         if msg == "OFF" and automacoes[topic]["timer_objeto"] is not None:
424             try:
425                 automacoes[topic]["timer_objeto"].cancel()
426                 automacoes[topic]["timer_objeto"] = None
427             except Exception:
428                 pass
429
430             try:
431                 mqttc.publish(topic, msg)
432             except Exception as err:
433                 print(f"Erro MQTT: {err}")
434
435     # --- PERSISTÊNCIA LOCAL ---
436     def salvar_dados_no_disco():
437         try:
438             dados_para_salvar = {
439                 "casa/rele1": {
440                     "timer_minutos": automacoes["casa/rele1"]["timer_minutos"],
441                     "hora_agenda": automacoes["casa/rele1"]["hora_agenda"],
442                     "dias_agenda": list(automacoes["casa/rele1"]["dias_agenda"])
443                 },
444                 "casa/rele2": {
445                     "timer_minutos": automacoes["casa/rele2"]["timer_minutos"],
446                     "hora_agenda": automacoes["casa/rele2"]["hora_agenda"],
447                     "dias_agenda": list(automacoes["casa/rele2"]["dias_agenda"])
448                 }
449             }
450             with open(CONFIG_FILE, "w") as f:
451                 json.dump(dados_para_salvar, f)
452             print("DEBUG: Dados guardados em segurança!")
453         except Exception as e:
454             print(f"Erro ao salvar dados: {e}")
455
456     # --- LÓGICA DO TEMPORIZADOR (TIMER) ---
457     def desligar_por_timeout(pino_rele, switch_componente):
458         try:
459             mqttc.publish(pino_rele, "OFF")
460             switch_componente.value = False
461             automacoes[pino_rele]["timer_objeto"] = None
462             automacoes[pino_rele]["timer_minutos"] = 0
463
464             page.snack_bar = ft.SnackBar(ft.Text(f"🔌 {pino_rele} desligado! Tempo terminou."))
465             page.snack_bar.open = True
466             page.update()
467         except Exception as e:
468             print(f"Erro no timeout do timer: {e}")
469
470     def acionar_temporizador(pino_rele, txt_timer, switch_componente):
471         try:
472             minutos = int(txt_timer.value)
473         except ValueError:
474             page.snack_bar = ft.SnackBar(ft.Text("Insira um número válido de minutos!"))
475             page.snack_bar.open = True
476             page.update()
477             return
478
479         if minutos <= 0:
480             page.snack_bar = ft.SnackBar(ft.Text("O tempo deve ser maior que zero!"))
481             page.snack_bar.open = True

```

```

482         page.update()
483         return
484
485         if automacoes[pino_rele]["timer_objeto"] is not None:
486             automacoes[pino_rele]["timer_objeto"].cancel()
487
488         mqttc.publish(pino_rele, "ON")
489         switch_componente.value = True
490
491         page.snack_bar = ft.SnackBar(ft.Text(f"⌚ Temporizador iniciado por {minutos}
492         minuto(s)"))
493         page.snack_bar.open = True
494         page.update()
495
496         segundos = minutos * 60
497         t = threading.Timer(segundos, desligar_por_timeout, args=[pino_rele,
498         switch_componente])
499         automacoes[pino_rele]["timer_objeto"] = t
500         automacoes[pino_rele]["timer_minutos"] = minutos
501         t.start()
502
503         salvar_dados_no_disco()
504
505         # --- INPUTS VISUAIS ---
506         txt_timer_d6 = ft.TextField(label="Minutos", value="30", width=90,
507         text_align="center", dense=True)
508         txt_hora_d6 = ft.TextField(label="HH:MM", value="18:00", width=90,
509         text_align="center", dense=True)
510         buttons_d6 = {}
511
512         txt_timer_d7 = ft.TextField(label="Minutos", value="30", width=90,
513         text_align="center", dense=True)
514         txt_hora_d7 = ft.TextField(label="HH:MM", value="18:00", width=90,
515         text_align="center", dense=True)
516         buttons_d7 = {}
517
518         switch_luz = ft.Switch(value=False, data="casa/rele1", on_change=mudar_rele)
519         switch_tomada = ft.Switch(value=False, data="casa/rele2", on_change=mudar_rele)
520
521         # --- CARREGA DADOS DO ARQUIVO ANTES DE CRIAR INTERFACE ---
522         def carregar_dados_local():
523             try:
524                 if os.path.exists(CONFIG_FILE):
525                     with open(CONFIG_FILE, "r") as f:
526                         dados_dict = json.load(f)
527                         print(f"DEBUG: Dados carregados: {dados_dict}")
528
529                     for pino in ["casa/rele1", "casa/rele2"]:
530                         if pino in dados_dict:
531                             automacoes[pino]["hora_agenda"] =
532                             dados_dict[pino].get("hora_agenda", "18:00")
533                             automacoes[pino]["timer_minutos"] =
534                             dados_dict[pino].get("timer_minutos", 0)
535                             dias_carregados = dados_dict[pino].get("dias_agenda", [])
536                             automacoes[pino]["dias_agenda"] = set(dias_carregados)
537             except Exception as e:
538                 print(f"Erro ao carregar dados: {e}")
539
540         carregar_dados_local()
541
542         # --- CONSTRUTOR DE BLOCOS ---
543         def criar_bloco_automacao(pino_rele, txt_timer, txt_hora, buttons_dict,
544         switch_componente):
545             def salvar_agenda(e):
546                 automacoes[pino_rele]["hora_agenda"] = txt_hora.value
547                 salvar_dados_no_disco()
548                 page.snack_bar = ft.SnackBar(ft.Text(f"Agenda de {pino_rele} salva para
549                 as {txt_hora.value}!"))
550                 page.snack_bar.open = True
551                 page.update()
552
553             def alternar_dia_btn(e):
554                 container_alvo = e.control.content

```



```

545     dia = container_alvo.data
546
547     if dia in automacoes[pino_rele]["dias_agenda"]:
548         automacoes[pino_rele]["dias_agenda"].discard(dia)
549         container_alvo.bgcolor = "transparent"
550         container_alvo.border = ft.Border(
551             ft.BorderSide(1, "white30"), ft.BorderSide(1, "white30"),
552             ft.BorderSide(1, "white30"), ft.BorderSide(1, "white30")
553         )
554     else:
555         automacoes[pino_rele]["dias_agenda"].add(dia)
556         container_alvo.bgcolor = "blue"
557         container_alvo.border = ft.Border(
558             ft.BorderSide(1, "blue"), ft.BorderSide(1, "blue"),
559             ft.BorderSide(1, "blue"), ft.BorderSide(1, "blue")
560         )
561
562     salvar_dados_no_disco()
563     page.update()
564
565     dias_semana = ["Seg", "Ter", "Qua", "Qui", "Sex", "Sáb", "Dom"]
566     lista_botoes = []
567
568     for d in dias_semana:
569         esta_selecionado = d in automacoes[pino_rele]["dias_agenda"]
570         bgcolor = "blue" if esta_selecionado else "transparent"
571         cor_borda = "blue" if esta_selecionado else "white30"
572
573         container_chip = ft.Container(
574             content=ft.Text(d, size=11, weight="bold", color="white"),
575             alignment="center",
576             padding=8,
577             width=45,
578             height=32,
579             border_radius=6,
580             border=ft.Border(
581                 ft.BorderSide(1, cor_borda), ft.BorderSide(1, cor_borda),
582                 ft.BorderSide(1, cor_borda), ft.BorderSide(1, cor_borda)
583             ),
584             bgcolor=bgcolor,
585             data=d
586         )
587
588         detector_clique = ft.GestureDetector(
589             content=container_chip,
590             on_tap=alternar_dia_btn
591         )
592
593         buttons_dict[d] = container_chip
594         lista_botoes.append(detector_clique)
595
596     txt_hora.value = automacoes[pino_rele]["hora_agenda"]
597     if automacoes[pino_rele]["timer_minutos"] > 0:
598         txt_timer.value = str(automacoes[pino_rele]["timer_minutos"])
599
600     return ft.ExpansionTile(
601         title=ft.Text("Configurar Temporizador e Agenda", size=13,
602             color="blue200"),
603         maintain_state=True,
604         controls=[
605             ft.Container(
606                 padding=10,
607                 bgcolor="black26",
608                 border_radius=8,
609                 content=ft.Column([
610                     ft.Row([
611                         ft.Icon(ft.Icons.TIMER, color="amber"),
612                         ft.Text("Desligar em:", weight="w500"),
613                         txt_timer,
614                         ft.IconButton(
615                             icon=ft.Icons.PLAY_ARROW_ROUNDED,
616                             icon_color="green",
617                             on_click=lambda e: acionar_temporizador(pino_rele,

```

```

        txt_timer, switch_componente)
617     )
618     ], alignment="spaceBetween"),
619
620     ft.Divider(height=10, color="white10"),
621
622     ft.Row([
623         ft.Icon(ft.Icons.SCHEDULE, color="blue"),
624         ft.Text("Ligar às:", weight="w500"),
625         txt_hora,
626         ft.IconButton(icon=ft.Icons.SAVE, icon_color="blue",
627             on_click=salvar_agenda)
628     ], alignment="spaceBetween"),
629
630     ft.Text("Dias da semana:", size=12, color="white60"),
631     ft.Container(
632         height=100,
633         content=ft.Row(lista_botoes, wrap=True,
634             alignment="center", spacing=5)
635     )
636 ], spacing=10)
637 )
638
639 # --- MONTAGEM DOS CARDS ---
640 card_luz = ft.Card(
641     content=ft.Container(
642         padding=12,
643         content=ft.Column([
644             ft.Row([
645                 ft.Row([ft.Icon(ft.Icons.LIGHTBULB, color="amber", size=28),
646                     ft.Text("Luz (D6)", size=18, weight="w500")]),
647                 switch_luz
648             ], alignment="spaceBetween"),
649             criar_bloco_automacao("casa/rele1", txt_timer_d6, txt_hora_d6,
650                 buttons_d6, switch_luz)
651         ])
652     )
653 )
654
655 card_tomada = ft.Card(
656     content=ft.Container(
657         padding=12,
658         content=ft.Column([
659             ft.Row([
660                 ft.Row([ft.Icon(ft.Icons.POWER, color="blueGrey200", size=28),
661                     ft.Text("Tomada (D7)", size=18, weight="w500")]),
662                 switch_tomada
663             ], alignment="spaceBetween"),
664             criar_bloco_automacao("casa/rele2", txt_timer_d7, txt_hora_d7,
665                 buttons_d7, switch_tomada)
666         ])
667     )
668 )
669
670 # --- RENDERIZAÇÃO DA INTERFACE ---
671 page.add(
672     ft.Container(height=15),
673     ft.Row([status_led, status_text], alignment="center"),
674     ft.Divider(),
675     card_luz,
676     card_tomada
677 )
678 page.update()
679
680 # --- PROCESSAMENTO ASSÍNCRONO EM BACKGROUND THREAD ---
681 def conectar_mqtt_e_agendar():
682     try:
683         mqttc.connect(MQTT_SERVER, MQTT_PORT, 60)
684         mqttc.loop_start()
685     except Exception:
686         pass

```

```
683
684 def loop_verificacao_horario():
685     while True:
686         agora = datetime.datetime.now()
687         dias_map = ["Seg", "Ter", "Qua", "Qui", "Sex", "Sáb", "Dom"]
688         dia_atual_str = dias_map[agora.weekday()]
689         hora_atual_str = agora.strftime("%H:%M")
690
691         for pino, dados in automacoes.items():
692             if dados["hora_agenda"] == hora_atual_str and dia_atual_str in
693                 dados["dias_agenda"]:
694                 mqttc.publish(pino, "ON")
695                 if pino == "casa/rele1":
696                     switch_luz.value = True
697                 if pino == "casa/rele2":
698                     switch_tomada.value = True
699                 page.update()
700
701             time.sleep(60)
702
703     threading.Thread(target=conectar_mqtt_e_agendar, daemon=True).start()
704     threading.Thread(target=loop_verificacao_horario, daemon=True).start()
705
706 ft.app(main)
707     time.sleep(60)
708
709     threading.Thread(target=conectar_mqtt_e_agendar, daemon=True).start()
710     threading.Thread(target=loop_verificacao_horario, daemon=True).start()
711
712 ft.app(main)
```